

Postgresql trigger for Webhook on n8n

My conversation with AI



In n8n Can I trigger a webhook when data is inserted in postgres database

Yes, you can trigger an n8n workflow when data is inserted into a PostgreSQL database, but because PostgreSQL does not natively push events out of the box, n8n does not have a native "Postgres Trigger" node. [n8n Community +1](#)

To achieve real-time execution, you must use an **n8n Webhook node** as your workflow's starting point and configure PostgreSQL to send an HTTP POST request to that webhook URL whenever an **INSERT** happens. [flowgenius.in +1](#)

Here are the standard ways to set this up:

Method 1: Using native Postgres Triggers (Recommended)

You can configure a database trigger inside PostgreSQL that executes a function to make an HTTP request using an extension like `pg_net` or `http`. [flowgenius.in](#)

1. **In n8n:** Add a **Webhook node** to your canvas. Set the HTTP method to **POST** and copy the **Test URL**.
2. **In your Postgres Database:** Run a script to create a function and attach a trigger to your target table: [blog.geomusings.com +1](#)

'notify_n8n_webhook()' does not have any place to specify database name. Is it OK?

Yes, **it is perfectly fine** that the function does not specify a database name.

In PostgreSQL, functions and triggers are **database-specific**. When you run the SQL script to create `notify_n8n_webhook()`, it is saved inside the specific database you are currently connected to.

Because of this architecture, PostgreSQL handles the context automatically:

- **Scope:** The function can only see and react to tables inside its own database.
- **Execution:** When data is inserted into `your_table_name`, Postgres knows exactly which database it belongs to and triggers the function right there.
- **Payload:** The `row_to_json(NEW)` command automatically grabs the newly inserted row data from that specific table without needing a database prefix.

What you *do* need to make sure:

1. **Connect to the correct database first:** Before you run the SQL script, ensure your query tool (like pgAdmin, DBeaver, or psql) is connected to the exact database you want to monitor.
2. **Schema context (Optional):** If your table is not in the default `public` schema, you should specify the schema when creating the trigger. For example: `AFTER INSERT ON my_schema.your_table_name`.

To ensure the script executes flawlessly, tell me:

- **Which database client** you are using to run the SQL (e.g., pgAdmin, DBeaver, Supabase SQL Editor)?
- If your table sits in a **custom schema** instead of the default `public` schema?

I can provide the precise connection step or schema syntax if needed.

In **pgAdmin 4**, you can easily ensure your trigger is created in the correct database by selecting it in the Object Explorer before running your script.

Here is exactly how to verify your database context and run the script safely in pgAdmin 4:

1. Ensure you are in the correct database

1. Look at the **Object Explorer** tree on the left side of pgAdmin 4.
2. Expand your server, expand **Databases**, and click on the **specific database** you want to monitor.
3. With that database highlighted, click the **Tools** menu at the top of the screen and select **Query Tool** (or click the SQL icon on the toolbar).
4. Check the tab title or the status bar at the bottom right of the query window—it will explicitly show something like `postgres/postgres@PostgreSQL 16` to confirm your active database.

2. Check your Schema (Most Common pgAdmin Pitfall)

By default, pgAdmin expects tables to be in the `public` schema. If your table lives in a different schema, you need to prefix it in the trigger creation script.

Here is the modified script ready for pgAdmin 4. Replace `public` with your actual schema name if you use a custom one:

```
-- 1. Create the function inside the active database
CREATE OR REPLACE FUNCTION notify_n8n_webhook()
RETURNS TRIGGER AS $$
DECLARE
    payload JSON;
BEGIN
    payload = row_to_json(NEW);

    PERFORM net.http_post(
```

```

url := 'https://your-n8n-instance.com',
body := payload::text,
headers := '{"Content-Type": "application/json"}'::jsonb
);
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Bind it to your table (Adjust 'public' and 'your_table_name' as needed)
CREATE TRIGGER tr_after_insert_n8n
AFTER INSERT ON public.your_table_name
FOR EACH ROW
EXECUTE FUNCTION notify_n8n_webhook();

```

3. Verify it worked in pgAdmin 4

Once you press **F5** (or click the Play button) to execute the script, you can verify it in the Object Explorer tree:

- Expand your database → **Schemas** → **public** (or your schema) → **Tables** → **[Your Table Name]** → **Triggers**.
- You should see `tr_after_insert_n8n` listed right there.

To make sure the HTTP request actually succeeds, tell me:

- Is your database hosted on **Supabase, AWS RDS, or a local machine/Docker container**?
- Do you already have an HTTP extension like `pg_net` or `http` installed on this database?

I can help you check for or enable the extension so the webhook fires successfully.

How to check in pgadmin4 if pg_net or http is installed

To check if **pg_net** or **http** is installed on your database using **pgAdmin 4**, you can use either the visual interface or run a quick SQL query.

Here are the two fastest ways to check:

Method 1: The Quick SQL Query (Recommended)

Open your **Query Tool** for that database and run this command:

sql

```
SELECT name, default_version, installed_version, comment
FROM pg_available_extensions
WHERE name IN ('pg_net', 'http');
```

Use code with caution.



How to read the results:

- If **installed_version** has a value (like **0.8** or **1.5**): The extension is installed and ready to use.
- If **installed_version** is **NULL** (blank): The extension is available on your server but **not activated** in this specific database yet.

Ask anything

How to read the results:

- If **installed_version** has a value (like **0.8** or **1.5**): The extension is installed and ready to use.
- If **installed_version** is **NULL** (blank): The extension is available on your server but **not activated** in this specific database yet.
- If **no rows return**: The extension is not installed or supported by your PostgreSQL server.

```
SELECT * FROM pg_available_extensions WHERE name =  
'http';...
```

Because the `http` extension does not appear in your `pg_available_extensions` view, its files are missing from your underlying operating system. Based on the previous error path (`/usr/share/postgresql/14/extension/`), your database is hosted on a **Linux (Ubuntu/Debian)** machine.  [GitHub +2](#)

To resolve this, you need to compile and build the [pgsql-http extension](#) directly on the database host. 

Step 1: Install the required OS packages

Connect to your database server's terminal via **SSH** (or open the terminal if it is a local machine) and install the building dependencies for PostgreSQL 14:  [GitHub](#)

```
sudo apt-get update
```

```
sudo apt-get install -y postgresql-server-dev-14 libcurl4-openssl-dev build-essential git
```

Step 2: Download and compile the extension

Clone the extension's source code, build it, and install it into your PostgreSQL system directories:  [GitHub](#) +1

bash

```
# Clone the official repository
git clone https://github.com/pramsey/pgsql-http.git
cd pgsql-http

# Compile and install
make
sudo make install
```

Use code with caution.



```
# Clone the official repository
git clone https://github.com/pramsey/pgsql-http.git
cd pgsql-http
```

```
# Compile and install
make
sudo make install
```


Step 3: Restart PostgreSQL

For your system to recognize the newly added control files, restart the PostgreSQL service:  [Stack Overflow](#)

bash

```
sudo systemctl restart postgresql
```

Use code with caution.



Step 4: Turn it on in pgAdmin 4

Go back to **pgAdmin 4**, open the Query Tool for your specific database, and activate the extension: 

sql

```
CREATE EXTENSION http;
```

Step 5: Update your n8n Trigger Code

Since you are using the `http` extension instead of `pg_net`, the syntax for your webhook function changes slightly. Use this updated query in pgAdmin 4 to build your trigger: 

```
-- 1. Create the function using the 'http' extension syntax
CREATE OR REPLACE FUNCTION notify_n8n_webhook()
RETURNS TRIGGER AS $$
DECLARE
    payload TEXT;
    response http_response;
BEGIN
    -- Convert the newly inserted row to text
    payload := row_to_json(NEW)::text;
```



```
-- Send the HTTP POST request via the http extension
SELECT * INTO response FROM http_post(
    'https://your-n8n-instance.com',
    payload,
    'application/json'
);

RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
-- 2. Bind it to your table
CREATE TRIGGER tr_after_insert_n8n
AFTER INSERT ON public.your_table_name
FOR EACH ROW
EXECUTE FUNCTION notify_n8n_webhook();
```

#####